

控制器部分设计

1. 指定的指令如下

指令	类型	opcode	function	含义
addu	R	000000	100001	将寄存器 rs 的值与寄存器 rt 的值相加，结果写入 rd 寄存器中。
sub	R	000000	100010	将寄存器 rs 的值与寄存器 rt 的值相减，结果写入 rd 寄存器中。不考虑溢出。
slt	R	000000	101010	将寄存器 rs 的值与寄存器 rt 中的值进行有符号数比较，如果寄存器 rs 中的值小，则寄存器 rd 置 1；否则寄存器 rd 置 0。 实现的时候可以实现位 rs-rt, 将符号位送 rd 的第 0 位。
lui	I	001111	无	将 16 位立即数 imm 写入寄存器 rt 的高 16 位，寄存器 rt 的低 16 位置 0。
ori	I	001101	无	寄存器 rs 中的值与 0 扩展至 32 位的立即数 imm 按位逻辑或，结果写入寄存器 rt 中。
bne	I	000101	无	如果寄存器 rs 的值不等于寄存器 rt 的值则转移，否则顺序执行。转移目标由立即数 offset 左移 2 位并进行有符号扩展的值加上该分支指令对应的延迟槽指令的 PC 计算得到。 实现的时候可以用 ALU 做减法，然后用是否为 0 标志位进行控制。
lw	I	100011	无	将 base 寄存器（rs 的位置）的值加上符号扩展后的立即数 offset 得到访存的地址，据此地址从存储器中读取连续 4 个字节的值，写入到 rt 寄存器中。 注意：测试程序中给出的地址是对齐的，所以不用考虑地址不对齐触发例外。
sw	I	101011	无	将 base 寄存器的值加上符号扩展后的立即数 offset 得到访存的虚地址，如果地址不是 4 的整数倍则触发地址错例外，否则据此虚地址将 rt 寄存器存入存储器中。 注意：测试程序中给出的地址是对齐的，所以不用考虑地址不对齐触发例外。
stp	其他	111111	无	所有控制信号清 0，时钟输入终止。如果是五段流水，要求 stp 指令也为五段，也就是 stp 指令退出流水线后，才能停止时钟输入。

2. 带控制信号的数据通路

控制可以采用两种设计方案，如下：

设计方案一：两阶段译码。（如书上给出的）

设计方案二：单阶段译码。（一次性译码）

请任选一种设计方案，参照书上给出的带控制信号的数据通路图，完成本实验任务的数据通路图。

注意：直接把书上的拿来是不对的！

3. 控制信号设计（我采用设计方案一）

若采用的是方案一，请从这里开始：

若采用的是设计方案二，请把设计方案一这部分模板删掉，直接保留设计方案二的部分。

（1）第二阶段译码（即 ALUop+Func，生成 ALU 控制信号）

指令	类型	操作码	ALUop	Func 字段	ALU 动作	ALU 控制信号
addu	R	000000	010	100001	加法	000
sub	R	000000	010	100010	减法	001
slt	R	000000	010	101010	比较	011
lui	I	001111	100	无	高位加载	100
ori	I	001101	011	无	逻辑或	010
bne	I	000101	001	无	减法	001
lw	I	100011	000	无	加法	000
sw	I	101011	000	无	加法	000
stp	其他	111111	000	无	无	000

（2）第一阶段译码（即 opcode，生成控制信号）

信号含义定义：

信号名	“0” 时定义	“1” 时定义
RegDst	写入寄存器的目标来自 rt	写入寄存器的目标来自 rd

Branch	不是分支指令	是分支指令
MemRead	无	存储器读
MemtoReg	写回寄存器的数据来自 ALU	写回寄存器的数据来自存储器
MemWrite	无	存储器写
ALUSrc	第二个 ALU 操作数来自 rt	第二个操作数来自 imm 的扩展
RegWrite	无	寄存器写
ALUop		
Extop	零扩展	符号扩展
Stop	正常	停机

针对指令的信号值：

指令	操作码	RegDst	ALUOp	ALUSrc	Branch	MemRead	MemWrite	RegWrite	MemtoReg	Extop	Stop
addu	000000	1	010	0	0	0	0	1	0	X	0
sub	000000	1	010	0	0	0	0	1	0	x	0
slt	000000	1	010	0	0	0	0	1	0	x	0
lui	001111	0	100	1	0	0	0	1	0	x	0
ori	001101	0	011	1	0	0	0	1	0	0	0
bne	000101	x	001	0	1	0	0	0	x	1	0
lw	100011	0	000	1	0	1	0	1	1	1	0
sw	101011	x	000	1	0	0	1	0	x	1	0
stp	111111	0	000	0	0	0	0	0	0	0	1

4. 控制器模块的设计（我采用设计方案一）

若采用的是方案一，请从这里开始；

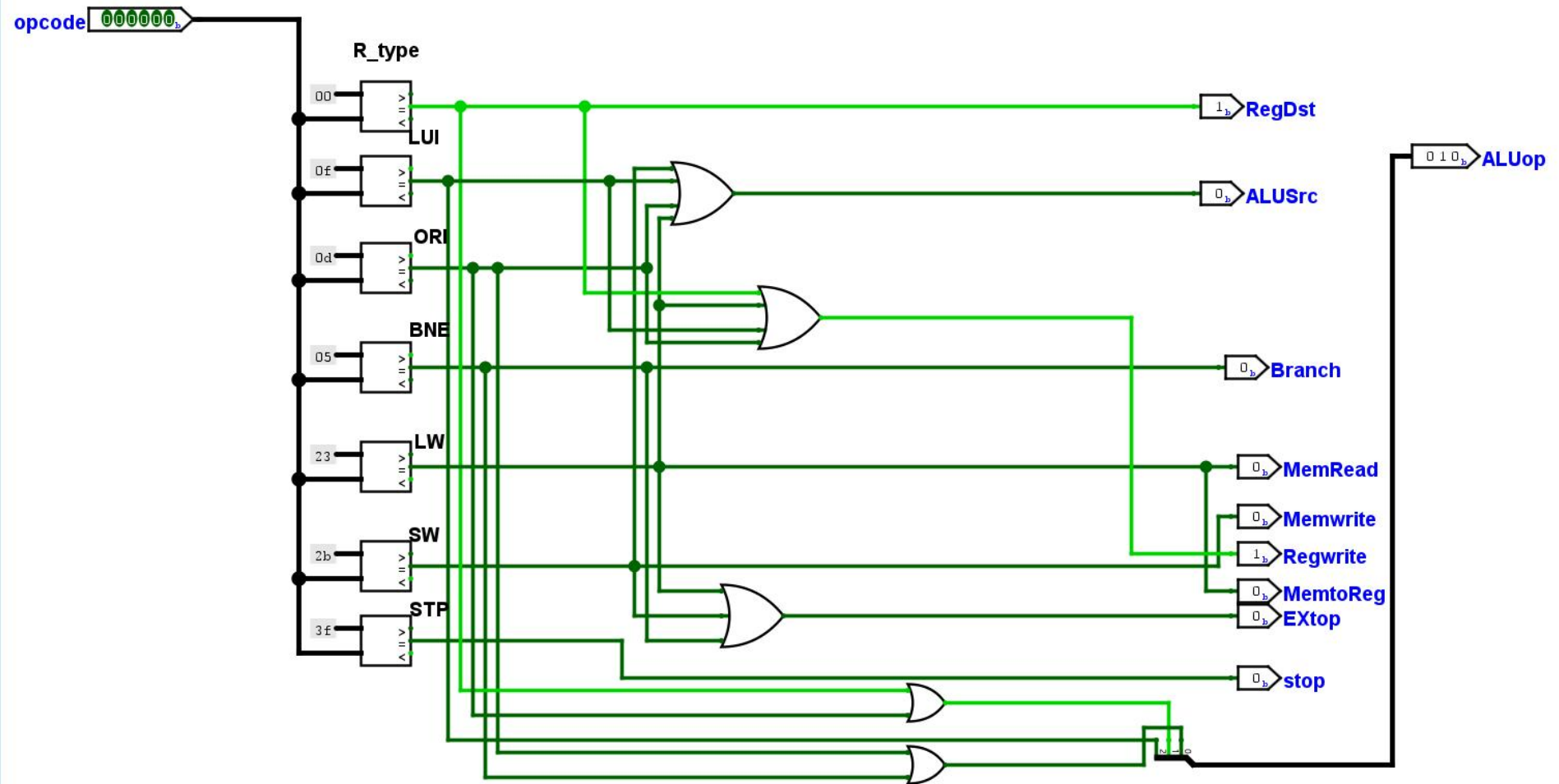
若采用的是设计方案二，请把设计方案一这部分模板删掉，直接保留设计方案二的部分。

（1）CU 模块

真值表：

输入	输出									
CU_in	RegDst	ALUOp	ALUSrc	Branch	MemRead	MemWrite	RegWrite	MemtoReg	Extop	Stop

设计截图：

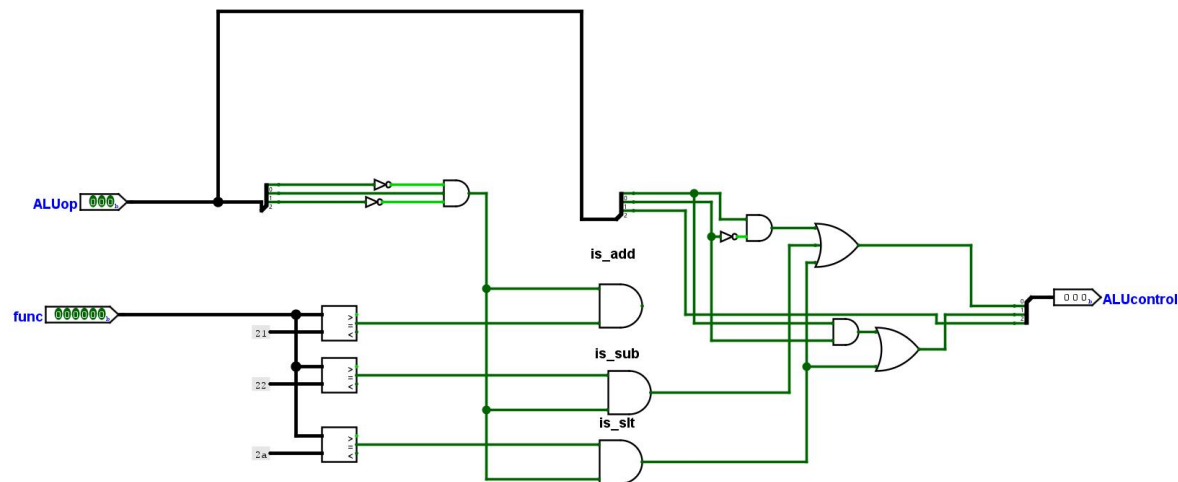


(2) ALUControl 模块

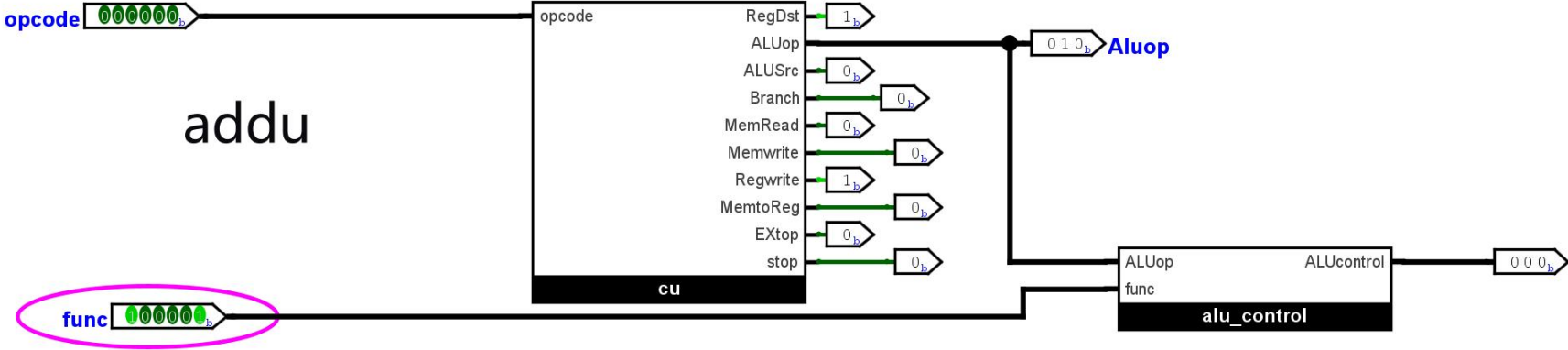
真值表:

输入		输出
ALUop	func	AluControl

设计截图:



(3) 如果需要其他模块, 请自己补充真值表和设计截图。



指令	类型	操作码	ALUOp	Func 字段	ALU 动作	ALU 控制信号
addu	R	000000	010	100001	加法	000
sub	R	000000	010	100010	减法	001
slt	R	000000	010	101010	比较	011
lui	I	001111	100	无	高位加载	100
ori	I	001101	011	无	逻辑或	010
bne	I	000101	001	无	减法	001
lw	I	100011	000	无	加法	000
sw	I	101011	000	无	加法	000
stp	其他	111111	000	无	无	000

针对指令的信号值:

指令	操作码	RegDst	ALUOp	ALUSrc	Branch	MemRead	MemWrite	RegWrite	MemtoReg	Extop	Stop
addu	000000	1	010	0	0	0	0	1	0	X	0
sub	000000	1	010	0	0	0	0	1	0	x	0
slt	000000	1	010	0	0	0	0	1	0	x	0
lui	001111	0	100	1	0	0	0	1	0	x	0
ori	001101	0	011	1	0	0	0	1	0	0	0
bne	000101	x	001	0	1	0	0	0	x	1	0
lw	100011	0	000	1	0	1	0	1	1	1	0
sw	101011	x	000	1	0	0	1	0	x	1	0
stp	111111	0	000	0	0	0	0	0	0	0	1

设计 模拟

- al_u_control
- main
 - cu
 - al_u_control
 - 线路
 - 逻辑门
 - 复用器
 - 运算器
 - 存储器
 - 输入/输出
 - TTL
 - TCL
 - BFH mega 函数
 - 额外输入/输出
 - SOC

属性 状态

引脚 "func"

是否支持 FPGA:

支持

方向

→东

输出?

否

数据位

6

三态?

否

驱动行为

不变

示数

func

示数字体

SansSerif 粗体 16

基数

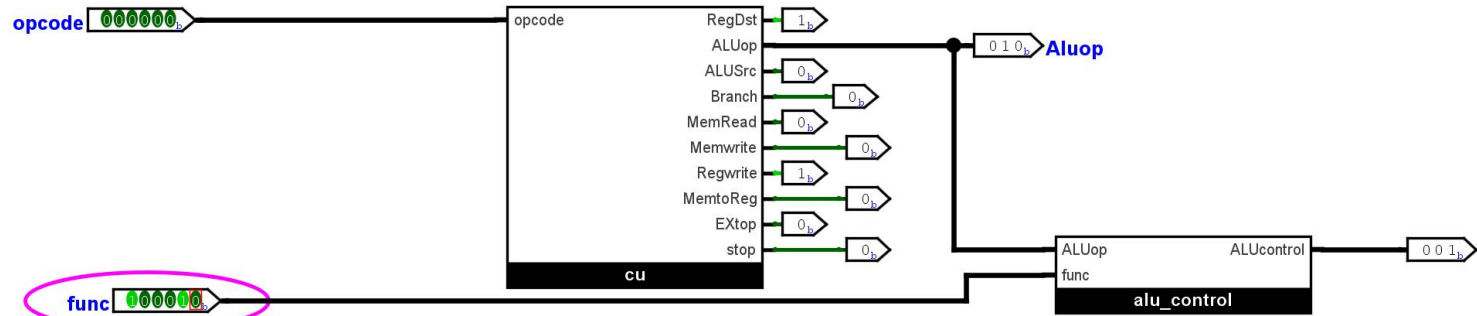
二进制

默认值:

0x0

外观

箭头形状

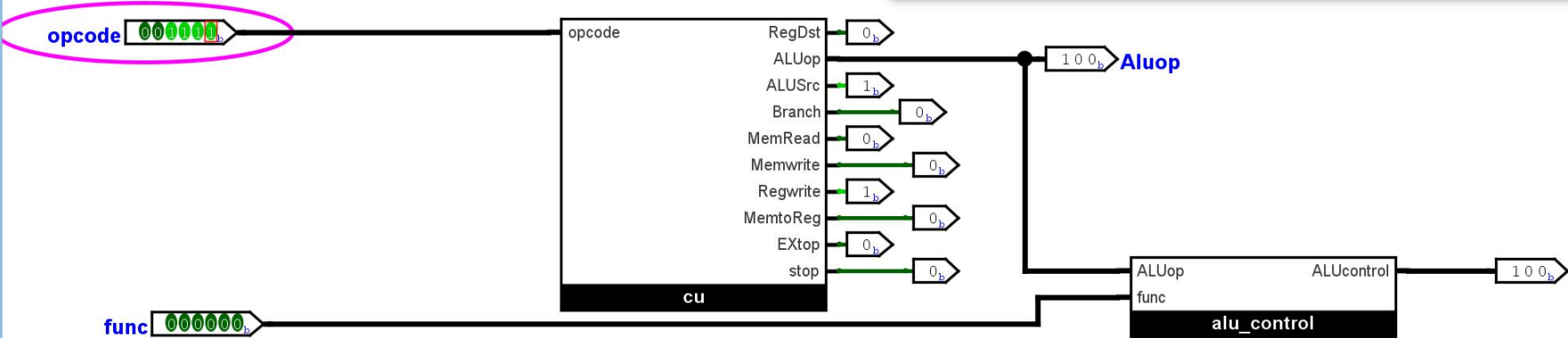
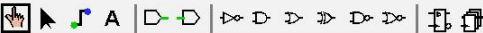


指令	类型	操作码	ALUOp	Func 字段	ALU 动作	ALU 控制信号
addu	R	000000	010	100001	加法	000
sub	R	000000	010	100010	减法	001
sllt	R	000000	010	101010	比较	011
lui	I	001111	100	无	高位加载	100
ori	I	001101	011	无	逻辑或	010
bne	I	000101	001	无	减法	001
lw	I	100011	000	无	加法	000
sw	I	101011	000	无	加法	000
stp	其他	111111	000	无	无	000

针对指令的信号值:

指令	操作码	RegDst	ALUOp	ALUSrc	Branch	MemRead	MemWrite	RegWrite	MemtoReg	Extop	Stop
addu	000000	1	010	0	0	0	0	1	0	X	0
sub	000000	1	010	0	0	0	0	1	0	x	0
sllt	000000	1	010	0	0	0	0	1	0	x	0
lui	001111	0	100	1	0	0	0	1	0	x	0
ori	001101	0	011	1	0	0	0	1	0	0	0
bne	000101	x	001	0	1	0	0	0	x	1	0
lw	100011	0	000	1	0	1	0	1	1	1	0
sw	101011	x	000	1	0	0	1	0	x	1	0
stp	111111	0	000	0	0	0	0	0	0	0	1

[illegible]

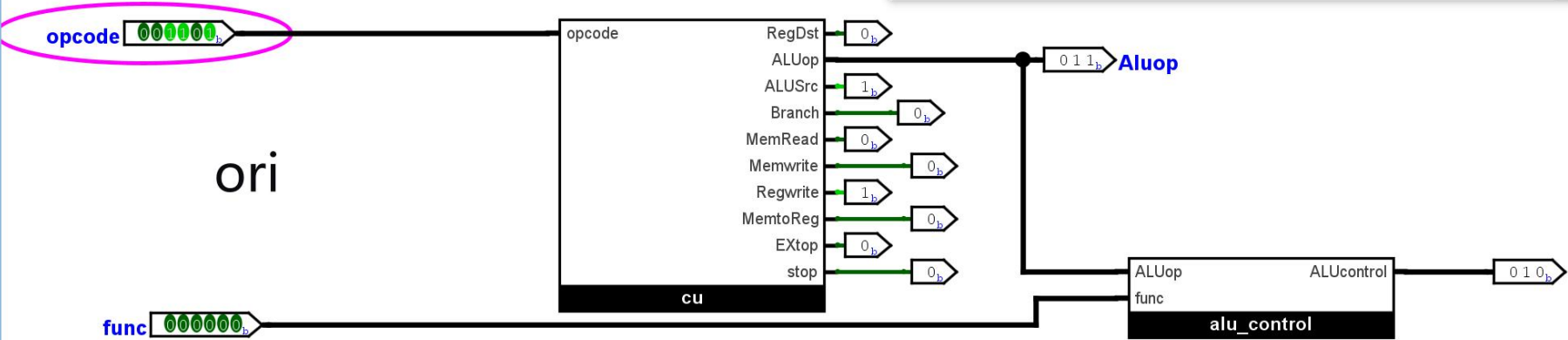
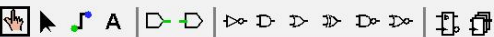


指令	类型	操作码	ALUOp	Func 字段	ALU 动作	ALU 控制信号
addu	R	000000	010	100001	加法	000
sub	R	000000	010	100010	减法	001
slt	R	000000	010	101010	比较	011
lui	I	001111	100	无	高位加载	100
ori	I	001101	011	无	逻辑或	010
bne	I	000101	001	无	减法	001
lw	I	100011	000	无	加法	000
sw	I	101011	000	无	加法	000
stp	其他	111111	000	无	无	000

持

针对指令的信号值:

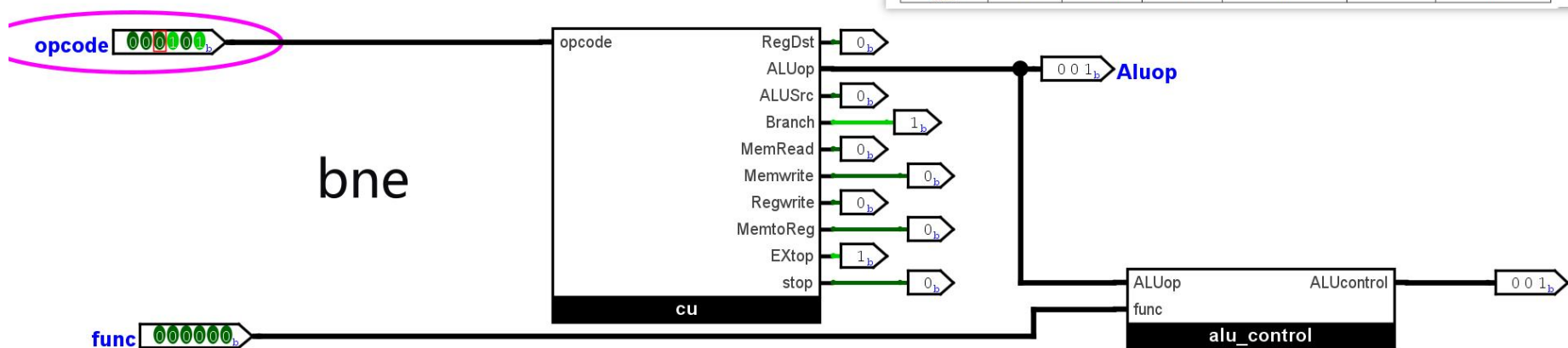
指令	操作码	RegDst	ALUOp	ALUSrc	Branch	MemRead	MemWrite	RegWrite	MemtoReg	Extop	Stop
addu	000000	1	010	0	0	0	0	1	0	X	0
sub	000000	1	010	0	0	0	0	1	0	x	0
slt	000000	1	010	0	0	0	0	1	0	x	0
lui	001111	0	100	1	0	0	0	1	0	x	0
ori	001101	0	011	1	0	0	0	1	0	0	0
bne	000101	x	001	0	1	0	0	0	x	1	0
lw	100011	0	000	1	0	1	0	1	1	1	0
sw	101011	x	000	1	0	0	1	0	x	1	0
stp	111111	0	000	0	0	0	0	0	0	0	1



指令	类型	操作码	ALUOp	Func 字段	ALU 动作	ALU 控制信号
<u>addu</u>	R	000000	010	100001	加法	000
<u>sub</u>	R	000000	010	100010	减法	001
<u>slt</u>	R	000000	010	101010	比较	011
<u>lui</u>	I	001111	100	无	高位加载	100
<u>ori</u>	I	001101	011	无	逻辑或	010
<u>bne</u>	I	000101	001	无	减法	001
<u>lw</u>	I	100011	000	无	加法	000
<u>sw</u>	I	101011	000	无	加法	000
<u>stp</u>	其他	111111	000	无	无	000

持

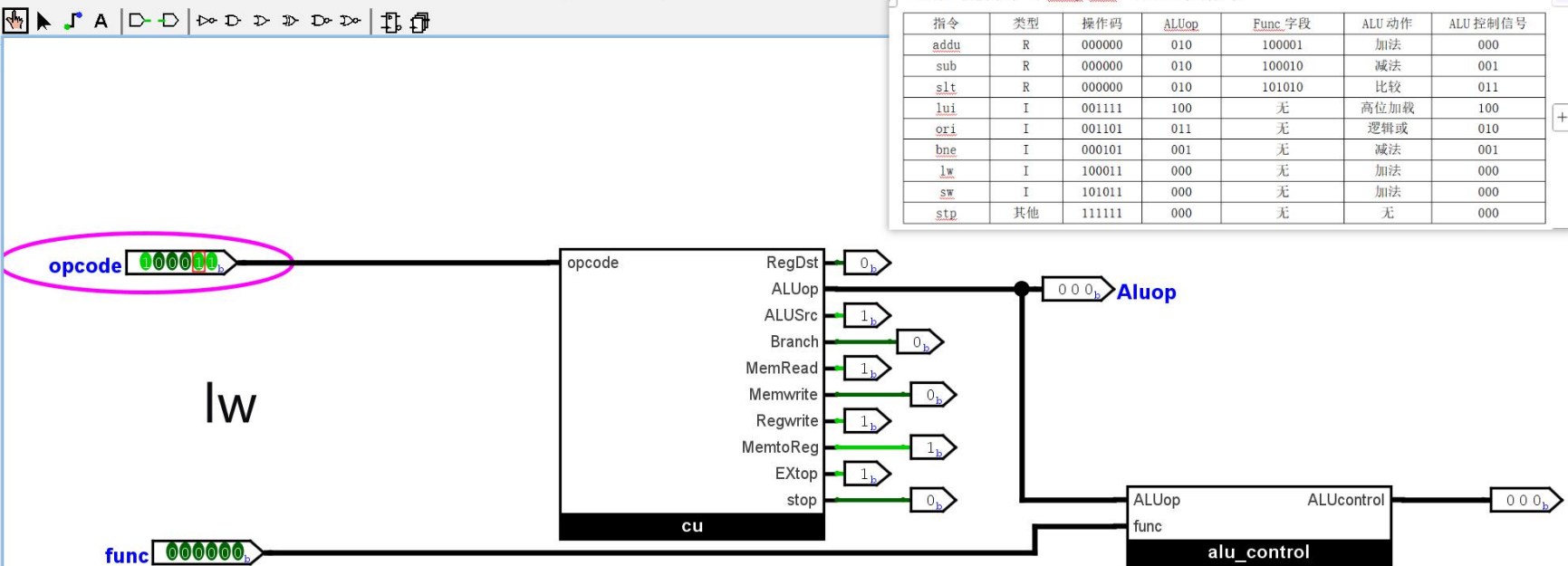
针对指令的信号值:											
指令	操作码	RegDst	ALUOp	ALUSrc	Branch	MemRead	MemWrite	RegWrite	MemtoReg	Extop	Stop
<u>addu</u>	000000	1	010	0	0	0	0	1	0	X	0
<u>sub</u>	000000	1	010	0	0	0	0	1	0	x	0
<u>slt</u>	000000	1	010	0	0	0	0	1	0	x	0
<u>lui</u>	001111	0	100	1	0	0	0	1	0	x	0
<u>ori</u>	001101	0	011	1	0	0	0	1	0	0	0
<u>bne</u>	000101	x	001	0	1	0	0	0	x	1	0
<u>lw</u>	100011	0	000	1	0	1	0	1	1	1	0
<u>sw</u>	101011	x	000	1	0	0	1	0	x	1	0
<u>stp</u>	111111	0	000	0	0	0	0	0	0	0	1



指令	类型	操作码	ALUop	Func 字段	ALU 动作	ALU 控制信号
<code>addu</code>	R	000000	010	100001	加法	000
<code>sub</code>	R	000000	010	100010	减法	001
<code>slt</code>	R	000000	010	101010	比较	011
<code>lui</code>	I	001111	100	无	高位加载	100
<code>ori</code>	I	001101	011	无	逻辑或	010
<code>bne</code>	I	000101	001	无	减法	001
<code>lw</code>	I	100011	000	无	加法	000
<code>sw</code>	I	101011	000	无	加法	000
<code>stp</code>	其他	111111	000	无	无	000

针对指令的信号值:

指令	操作码	RegDst	ALUOp	ALUSrc	Branch	MemRead	MemWrite	RegWrite	MemtoReg	Extop	Stop
<code>addu</code>	000000	1	010	0	0	0	0	1	0	X	0
<code>sub</code>	000000	1	010	0	0	0	0	1	0	x	0
<code>slt</code>	000000	1	010	0	0	0	0	1	0	x	0
<code>lui</code>	001111	0	100	1	0	0	0	1	0	x	0
<code>ori</code>	001101	0	011	1	0	0	0	1	0	0	0
<code>bne</code>	000101	x	001	0	1	0	0	0	x	1	0
<code>lw</code>	100011	0	000	1	0	1	0	1	1	1	0
<code>sw</code>	101011	x	000	1	0	0	1	0	x	1	0
<code>stp</code>	111111	0	000	0	0	0	0	0	0	0	1



le"

支持

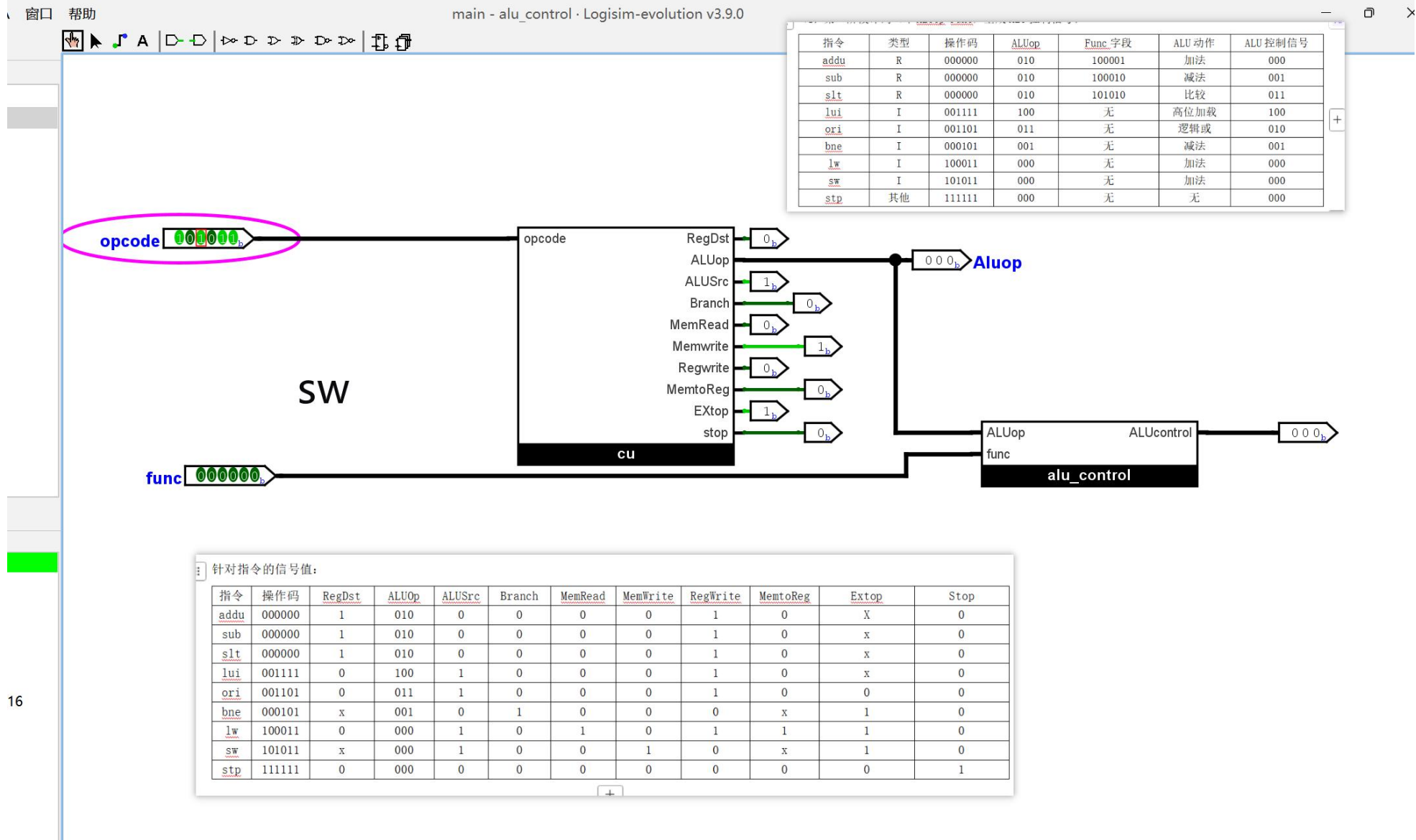
de

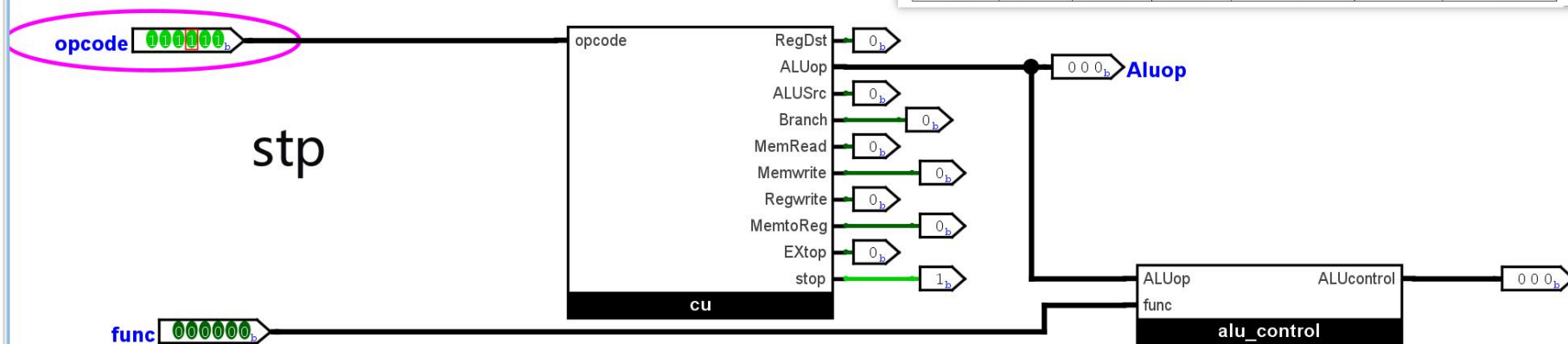
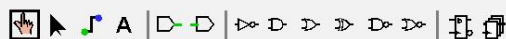
Serif 粗体 16

制

形状

针对指令的信号值:											
指令	操作码	RegDst	ALUOp	ALUSrc	Branch	MemRead	MemWrite	RegWrite	MemtoReg	Extop	Stop
addu	000000	1	010	0	0	0	0	1	0	X	0
sub	000000	1	010	0	0	0	0	1	0	x	0
slt	000000	1	010	0	0	0	0	1	0	x	0
lui	001111	0	100	1	0	0	0	1	0	x	0
ori	001101	0	011	1	0	0	0	1	0	0	0
bne	000101	x	001	0	1	0	0	0	x	1	0
lw	100011	0	000	1	0	1	0	1	1	1	0
sw	101011	x	000	1	0	0	1	0	x	1	0
stp	111111	0	000	0	0	0	0	0	0	0	1





指令	类型	操作码	ALUOp	Func 字段	ALU 动作	ALU 控制信号
<code>addu</code>	R	000000	010	100001	加法	000
<code>sub</code>	R	000000	010	100010	减法	001
<code>slt</code>	R	000000	010	101010	比较	011
<code>lui</code>	I	001111	100	无	高位加载	100
<code>ori</code>	I	001101	011	无	逻辑或	010
<code>bne</code>	I	000101	001	无	减法	001
<code>lw</code>	I	100011	000	无	加法	000
<code>sw</code>	I	101011	000	无	加法	000
<code>stp</code>	其他	111111	000	无	无	000

针对指令的信号值:

指令	操作码	RegDst	ALUOp	ALUSrc	Branch	MemRead	MemWrite	RegWrite	MemtoReg	Extop	Stop
<code>addu</code>	000000	1	010	0	0	0	0	1	0	X	0
<code>sub</code>	000000	1	010	0	0	0	0	1	0	x	0
<code>slt</code>	000000	1	010	0	0	0	0	1	0	x	0
<code>lui</code>	001111	0	100	1	0	0	0	1	0	x	0
<code>ori</code>	001101	0	011	1	0	0	0	1	0	0	0
<code>bne</code>	000101	x	001	0	1	0	0	0	x	1	0
<code>lw</code>	100011	0	000	1	0	1	0	1	1	1	0
<code>sw</code>	101011	x	000	1	0	0	1	0	x	1	0
<code>stp</code>	111111	0	000	0	0	0	0	0	0	0	1

数据通路图:

